Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 246 (2024) 381–390

Procedia Computer Science

www.elsevier.com/locate/procedia

28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024)

Review-Based Bot Smell Classification in Robotic Process Automation

Hiroyuki Nakagawa[a,b], Soshi Nitta[a], Tatsuhiro Tsuchiya[a]

[a] *Osaka University, 1-5 Yamadaoka, Suita, Osaka 565-0871, Japan*
[b] *Okayama University, 3-1-1 Tsushima-naka, Kita-ku, Okayama 700-8530 Japan*

Abstract

Robotic process automation (RPA) is a technique for automating desktop tasks by developing *bots*. While RPA enables automatic repetition of tasks, various defects can occur during actual operations. Some defects manifesting as *smells* in bot codes during the development phase are considered to possess distinct characteristics compared to conventional programs. This study aims to classify these smells in RPA and facilitate their detection. We first categorize RPA smells using code reviews that highlight smells in actual bot development. Based on the categorization, we also develop a preliminary smell detection tool. The categorization result reveals that the most prominent category is *substitutable process*, highlighting the importance of replacing a sequence of commands with single RPA commands. Experimental results show that the detection tool can find smells with a high degree of accuracy. This high accuracy seems to be attributable to the stronger constraints present in RPA code compared to conventional programming code.

Keywords: RPA; smells; code reviews; classification; NLP.

1. Introduction

Robotic process automation (RPA) [2][14] is a business automation technique designed to automate simple tasks, such as data copying and pasting, or input, by creating processes called *bots*. Bots simulate human activity and operate on the user interface (UI) of other applications [15]. Therefore, RPA has the potential to substitute rule-based tasks performed by humans [5][9]. Additionally, in contrast to conventional software development, RPA enables no-code or low-code development [16]. Given that RPA is designed for business process automation, it provides a large assortment of related commands and robot parts — componentized functions — for bot development. This substitution can

* Hiroyuki Nakagawa. Tel.: +81-862-51-7381; fax: +81-862-51-8256.
E-mail address: h-nakagawa@okayama-u.ac.jp

1877-0509
10.1016/j.procs.2024.09.417

enhance productivity, efficiency, and task execution accuracy [3] [10]. As a result, various sectors such as IT, insurance, and human resources have adopted RPA [6] [18] [17].

However, several defects have been reported in real-world RPA operations. These defects manifest as symptoms such as insufficient code or conditions, inappropriate encapsulation, and overly complex structures. Referred to as *smell* in this study, these symptoms seem to differ from those appearing in software developments using traditional programming languages. This distinction arises due to the unique nature of RPA, which is characterized by business automation, external application UI manipulation, and no-code development.

In this study, we classify smells in RPA based on code reviews, which serve as resources for identifying smells and highlighting code that directly causes a defect or could potentially lead to one. We define classification categories such as access violation, lack of process, and substitutable process based on the types of smells. We confirm the validity of these categories by applying them to test data and comparing the results with those from existing studies. We also implement a preliminary tool for detecting smells based on code similarity. Despite its simplicity, this tool achieves relatively high accuracy due to the distinct characteristics of RPA development.

The remainder of this paper is organized as follows: Section 2 presents related work and the aim of this study. Section 3 details our classification results of bot smells, while Section 4 describes the tool for detecting smells based on similarity. Finally, Section 5 concludes this study and outlines future research.

2. Background

The occurrence of defects in RPA represents a significant issue, a subject addressed in numerous studies. Schuler et al. [11] suggested that for developing a stable bot, elements like exception handling, work queues, and data management are crucial. Exception handling, in particular, emphasized by Syed et al. [19] as being vital, is closely related to the bot code and the coding phase, which are the focal points of this study. Furthermore, Egger et al. [1] proposed a method using logs to aid in bot modifications, wherein bot logs and process logs are merged for process mining. This amalgamated log provides an integrated perspective for bot development.

The goal of this study is to decrease the number of defects in RPA by identifying bot smells, and subsequently implementing an automatic detection tool for some of these elements. As RPA is a relatively new technology with an immature bot development procedure, it is a conducive environment for the inclusion of bot smells. Despite this, scant research has been conducted on potential smells present in bots. In this study, we systematize smells in RPA by classifying data obtained from actual RPA development. Additionally, we test a detection method appropriate for identifying some RPA smells and implement an automatic smell detection tool.

This paper makes three main contributions. (1) The results of this classification can serve as a standard for creating bots, taking into account the code smells that need to be addressed. (2) The classification results can be utilized as a benchmark for exhaustive detection during code reviews. Both measures are expected to reduce the number of defects in RPA. (3) The automated detection tool lowers debugging costs, enabling resources to be allocated to quality improvement at the bot creation stage, and tracking hard-to-detect defects, among other tasks.

3. Classification

3.1. Process

The classification process is conducted using a data-driven, bottom-up methodology, with reference to the defect classification study of Rothermel et al. [7]. This process comprises the following steps: (1) Divide the target defects into a training set and test set; (2) Determine the cause of each defect in the training set; (3) Identify the categories of defects; (4) Assign descriptive labels to these categories; (5) Arrange the labels into hierarchies based on category similarities; (6) Define the hierarchical categories of classification; and (7) Validate the classification using the test set.

This study leveraged 87 bots created by a software development company using Automation Anywhere [4] as an RPA development tool. Automation Anywhere is one of the most widely used RPA tools. These bots performed a variety of tasks, ranging from downloading data using an HTML browser to inputting data into a spreadsheet. Subsequently, a developer from the RPA tool development company reviewed the bots and pinpointed the codes causing defects. As depicted in Figure 1, the review includes the names of the target bots, line numbers, and code review comments.

ID	Bot Name	Line	Code Review Comments
0	Check Accounting	18	For more stable operation, it is recommended to set the KeyStroke delay slightly longer. Please adjust the delay according to the number of keystrokes you plan to input.
1	Request StockList	43-46	The hierarchy of the If statement is deep. Therefore, if the conditions after Line N are evaluated first, the hierarchy of the contents currently within the If statement can be elevated, thereby improving readability.
			⋮

Fig. 1. Example of reviews.

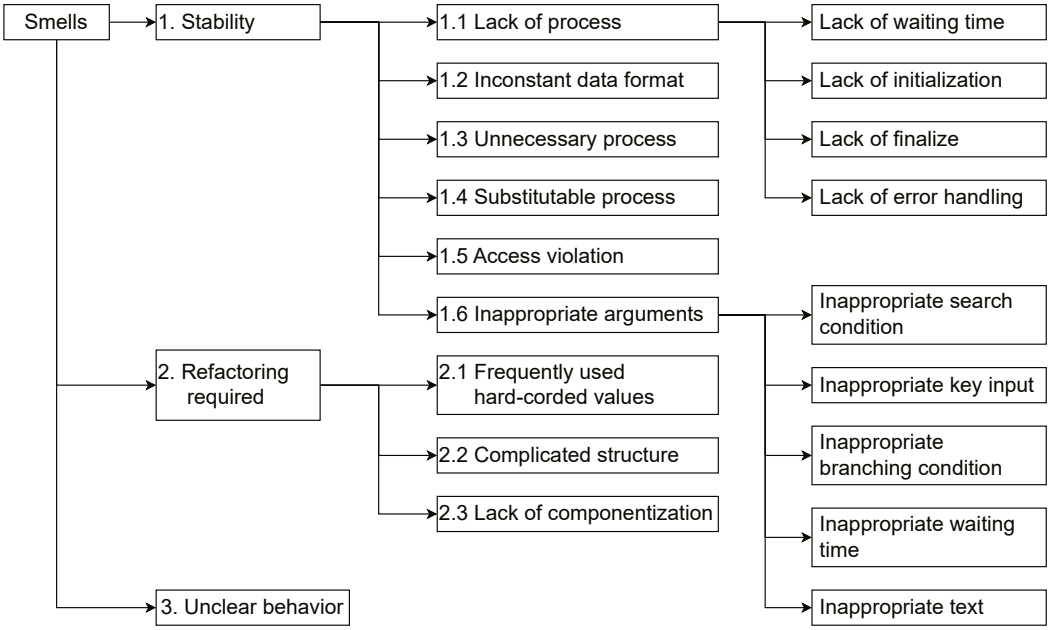


Fig. 2. A classification of RPA smells.

Using the above data, the following procedure is employed to classify the data. Initially, the reviews are divided into two parts: two-thirds for the training set data and one-third for test set data. Step 2 had already been executed with the code review comments. Thus, the next step involves identifying and classifying the categories of smells based on the review comments. The labels are then arranged into hierarchies based on their similarities. For instance, review comment 1, "*It is better to set the KeyStroke delay a little longer for a more stable operation. Please adjust the delay according to the number of keystrokes you want to input,*" was noted as an argument of the DELAY command; thus, the review is classified as an *inappropriate value of waiting time*. The category is then aggregated into a parent category named *inappropriate argument* in the hierarchization step. The classification is then tested using the test set to confirm that all reviews can be classified into categories.

3.2. Results

Figure 2 illustrates the established hierarchical classification. Following the classification process, the ensuring categories of smell causes were constructed:

- *1.1 Lack of process*: Necessary processing such as initialization, error handling, waiting for processing, etc. is missing.
- *1.2 Inconsistent data format*: The same type of data, like dates, is used in different formats.
- *1.3 Unnecessary process*: Unnecessary commands and processes exist, such as opening the same file or handling exceptions in places where errors cannot occur.

- *1.4 Substitutable process*: The process can be replaced by a few commands or robot parts that are more stable.
- *1.5 Access violation*: A value is assigned to a read-only variable.
- *1.6 Inappropriate argument*: Values for arguments, such as object search conditions, and waiting time settings, are inappropriate.
- *2.1 Frequently used hard-coded values*: Frequently changing values are coded directly.
- *2.2 Complicated structure*: Excessively complex branches or blocks that reduce readability and maintainability.
- *2.3 Lack of componentization*: Similar processes exist in multiple places, leading to redundant code.
- *3 Unclear behavior*: The bot behavior does not seem to align with the requirements.

Table 1 presents the number of reviews classified into each category. Regarding the testing trials of the classifiers, no reviews were unclassifiable using the classifiers created in the training set. If any reviews in the test set were unclassifiable, it would be necessary to create new categories. This approach is based on the work of Mika V. Mäntylä [26], who also classified code reviews. For each category of classification results, the ratio of the number of cases in the training set to the number of cases in the test set varied. This variation can be attributed to the small total number of cases in the training and test sets. Although the division between the training and test sets was made uniformly, the number of cases in each category differed. This difference is likely due to the unique characteristics and complexity of each case.

3.3. Discussion of Results

The *substitutable process* category is the most prevalent, largely because RPA is a specialized technique for desktop tasks. Numerous desktop tasks can be accomplished using individual RPA commands designed for file manipulation, spreadsheet operations, and key inputs. These commands and parts often prove more stable than the code implemented by RPA tool users for similar purposes. As such, numerous reviews have suggested replacing a series of commands with a single, more stable command.

Next, we discuss categories that are especially noteworthy compared to general programs. The category *inappropriate condition for object search* is distinctive. In a typical program, the API defines the protocol for interaction with external programs. However, in RPA, interactions occur with the UI that humans operate. It necessitates the bot developer to define conditions for specifying the UI and windows, which are potential defect inducers. This factor is unique to RPA, which is highly dependent on external applications.

The categories *lack of error handling* and *unnecessary process*, which contain reviews about excessive error handling, are also notable. Error handling is a critical issue when bots serve as human substitutes. Bots are robotic agents [25] that do not learn; hence, exception handling must be defined at the design stage.

Moreover, if the business process to be automated is not properly selected or well-modeled, frequent errors will ensure. However, error dialogs and other outputs from external applications are presumed to be interpreted by humans. Thus, when a bot processes an error, it must recognize and interpret the error. Error handling is thus a significant but challenging issue in RPA.

Other notable categories pertain to waiting time, namely *lack of waiting time* and *inappropriate value of waiting time*. The combined percentages of these two categories amounted to 12.8%. These categories arise because RPA bots interact with external applications such as browsers, spreadsheets, and mailers. It is crucial to issue the next instruction only after the successful completion of preceding commands is confirmed. Many reviews have highlighted the need for adequate command waiting times.

The influence of the dataset on the reproducibility of the classification results is also considered. Factors that affect classification results include the developer and their organization, development tools, and reviewers. The bots reviewed in this study were developed by 17 different individuals, suggesting the idiosyncrasies of individual development techniques are likely averaged out and not reflected in the overall results. Nonetheless, latent factors such as organizational development rules may have influenced the results. Although a specific RPA tool was used in this study, similar results might be expected with different RPA tools as many of the categories involve smells that are generally difficult to detect with RPA tool functions. However, easily detectable categories like access violation might depend on the RPA tool. Finally, regarding the reviewers, as they are as varied as the bot developers, dependence on them is deemed to be minimal. These findings suggest a reasonable degree of reproducibility in the classification results. However, as mentioned earlier, latent factors such as corporate culture may have influenced the classification results.

Table 1. Results of review classification. Labels "TR" and "TE" represent the number of reviews classified in the categories in the training set and testing set, respectively.

Category	TR	TE	Total	Rate
1.1.1 Lack of waiting time	21	10	31	0.059
1.1.2 Lack of initialize	13	7	20	0.038
1.1.3 Lack of finalize	2	4	6	0.011
1.1.4 Lack of error handling	6	9	15	0.029
1.2 Inconsistent data format	36	22	58	0.111
1.3 Unnecessary process	35	19	54	0.103
1.4 Substitutable process	81	39	120	0.229
1.5 Access violation	22	12	34	0.065
1.6.1 Inappropriate search condition	6	1	7	0.013
1.6.2 Inappropriate key input	9	1	10	0.019
1.6.3 Inappropriate branching condition	4	5	9	0.017
1.6.4 Inappropriate waiting time	21	15	36	0.069
1.6.5 Inappropriate text	12	8	22	0.038
2.1 Frequently used hard-coded values	18	6	24	0.046
2.2 Complicated structure	17	2	19	0.036
2.3 Need componentization	16	7	23	0.044
3 Unclear behavior	29	8	37	0.071
Total	348	175	523	1.000

Therefore, in the next section, the validity of the classification results will be verified by comparing them with results obtained in similar studies.

4. Smell Detection Tool

This section discusses automatic detection of categories presented in the previous section. Elements required for detection likely vary depending on the category. Hence, we focus on categories considered easy to detect and implement a preliminary smell detection tool that utilizes code reviews. Specifically, the target categories are *access violation*, *substitutable process*, and *inappropriate waiting time* because their detection does not necessitate parsing (dependent on the RPA tools used) or extensive data-driven contextual interpretation.

4.1. Overview of the tool

We utilize natural language processing (NLP) and machine learning for implementing detection methods. Defect detection using machine learning techniques, such as deep learning, is a rapidly evolving field. According to Akimova's survey [20], it is generally accepted that deep belief networks [21], convolutional neural networks [22], long short-term memory [23], and transformers [24] are applicable. Using these models for smell detection promises high accuracy but requires significant investment in implementation and training. However, RPA codes are difficult to generalize due to their dependence on the RPA tool used, and given the field's novelty, not many codes or data are available for training purposes. Therefore, we aim to implement and utilize a model that incurs relatively low implementation and training costs and employs a simple, low-cost similarity calculation method.

The tool operates under the assumption that a category of the classification to be detected is specified, and it detects lines similar to the lines flagged by the reviews belonging to that category. We implement two similarity calculation methods. The first employs NLP models: Word2Vec [13] and Word Mover's Distance [8]. The second method identifies words closely related to a particular smell, termed *feature words*, and calculates similarity using the overlap coefficient. We implement different calculation methods because the factors emphasized vary depending on the type of smell. For instance, in the case of *substitutable process*, the entire code section highlighted in the review pertains to the smell. However, for some smells, certain keywords hold important relevance. For example, in the case of *access violation*, which involves assignment to a read-only variable, the name of the protected variable is a keyword. In the case of *inappropriate value of waiting time*, the command name of the wait instruction becomes the keyword. Consequently, it is necessary to implement a detection method based on the type of smell to be detected.

The tool utilizes the following data:

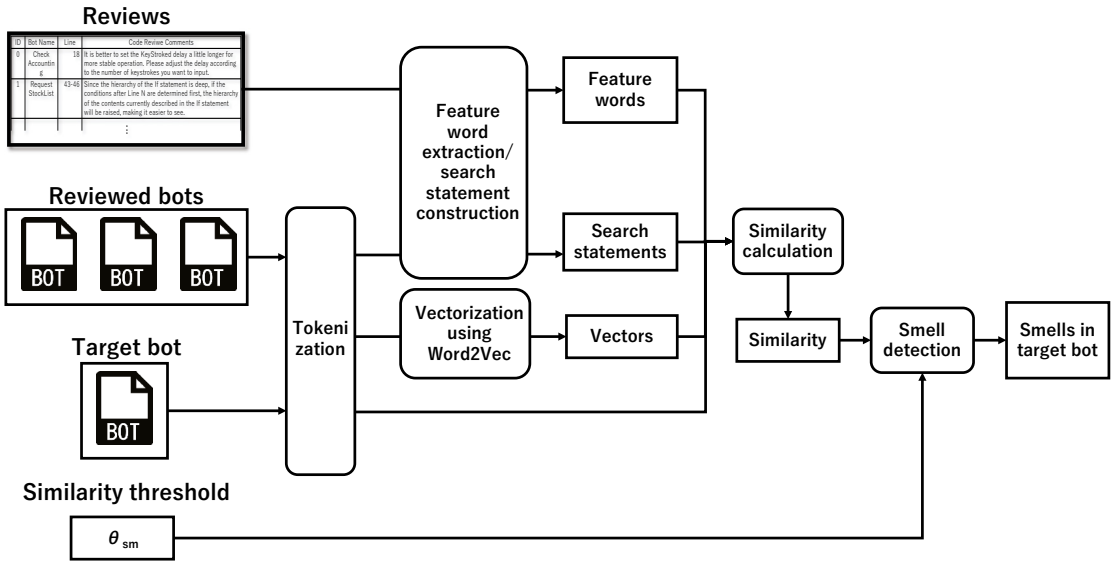


Fig. 3. An overview of the smell detection tool.

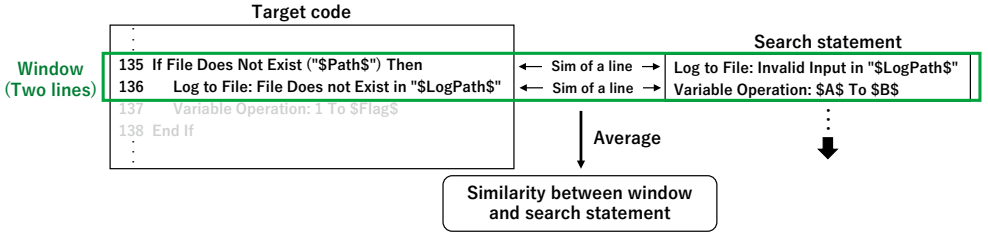


Fig. 4. Similarity calculation using window-base matching.

- Target bot: The bot to be checked for containing smells.
- Reviews: Reviews that highlight a smell to be detected.
- Reviewed bots: Bots on which the reviews concentrate.
- θ_{sm} ($\in [0, 1]$): Similarity threshold used to determine whether the code contains a smell or not.

Figure 3 presents an overview of this tool. The tool performs the following steps:

1. Tokenization. A statement is segmentalized into words. If a command name contains spaces, its words are concatenated to a word. Numerical values are replaced with corresponding tokens after equivalence partitioning.
2. Vectorization using Word2Vec.
3. Feature word extraction. Feature words are words closely related to a particular smell. These words are used for similarity calculation.
4. Search statement construction. A *search statement* includes one or more lines from the reviewed bot. For example, the search statement corresponding to the first review in Figure 1 is line 18 in a bot named *CheckAccounting*.
5. Similarity calculation between the search statement and lines in the target bot. If the search statement contains multiple lines, the tool calculates the similarity for each line. The similarity of multiple lines is defined as the average of these similarities, as depicted in Figure 4.
6. Repeat Steps 4 and 5, shifting lines of the target bot.
7. Repeat Steps 3 to 6 for all reviews.
8. Smell detection by comparing similarity values and the threshold θ_{sm}.

Values in codes	5 ms	40 ms	80 ms	100 ms	1000 ms
Tokens	TOKEN[Time-10]	TOKEN[Time-40]	TOKEN[Time-100]		TOKEN[Time100-]

Fig. 5. Example of partitioning for delay time.

In Step 1, numerical values in codes are replaced with tokens after equivalence partitioning. This software testing technique divides data into partitions of equivalent data from which test cases can be derived. In this study, the technique is used for classifying values, such as waiting times, into a small set of categories. When a tool directly handles various different numerical values, the similarity between essentially similar codes containing close but different values will become small. Therefore, values within a certain range and causing the same defects are classified into the same category to increase similarity and improve accuracy. This partitioning, however, has difficulty in defining boundary values. Since the boundary varies depending on the environment, it cannot be determined at the design time. Therefore, we define the boundary values from the values in bots. For example, from the code "*Keystrokes: [ENTER] in Browser with delay: 100ms,*" the value "100ms" would be extracted. If the values 5ms, 40ms, 80ms, 100ms, and 1000ms are obtained, the partitioning will be conducted as in Figure 5. After the partitioning, statements are vectorized using Word2Vec. Generated vectors are used for calculating similarity. We use Word Mover's Distance [27] for calculating similarity between two vectors.

Feature words are words closely related to a particular smell. For example, for the *inappropriate argument* category, the name of the command that receives arguments is a feature word. For the *access violation* category, the name of a variable that cannot be written or read is a feature word. To derive these feature words, a set of lines, hereafter referred to as a *group*, indicated by reviews that contain similar smells is used. Words that frequently appear in codes within a group are considered feature words closely related to the smells represented by that group. Therefore, the following equation is used to derive the feature words:

$$T_G = \left\{ t \mid \frac{|G_t|}{|G|} \leq \theta_G \right\}, \tag{1}$$

where T_G is a set of feature words, t is a word, G is a group, G_t is a set of codes in which word t appears, and θ_G is the threshold, which is a real number that falls within the range [0.0, 1.0].

Next, similarity is calculated using feature words and a window as a set of words. Feature words are terms closely related to a certain smell. Therefore, when many types of feature words appear in a window, the connection between the window and the smell is reinforced, thereby elevating the similarity. Hence, the following two preconditions must be satisfied. First, the similarity should be contingent upon the number of distinct types of feature words used in the window. Second, the maximum value is attained when all the feature words are used in the window. These preconditions are satisfied by the overlap coefficient [12], which defines the similarity between two sets. The overlap coefficient assumes a high value when two sets share more common elements, reaching a maximum value of 1.0 if one set is a subset of the other. The overlap coefficient between sets A and B is computed using the following equation:

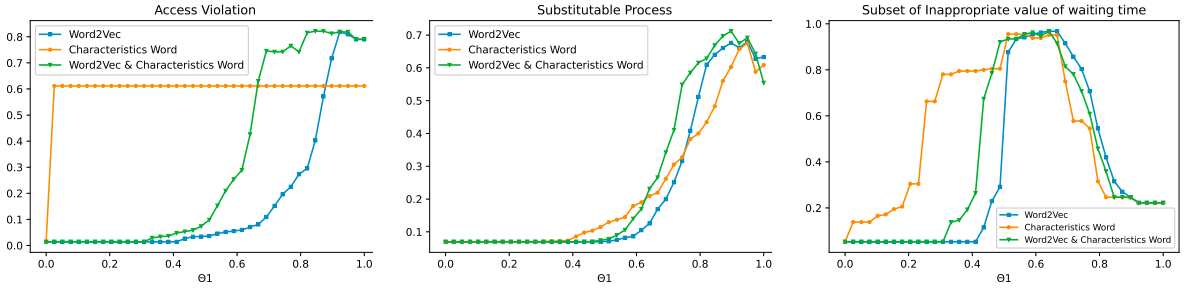
$$Overlap\ coefficient = \frac{|A \cap B|}{min(|A|, |B|)} \tag{2}$$

In many cases, the count of elements in the feature words is at most equivalent to the number of words in the window. This number is small in comparison to the total number of words in the window. Therefore, the similarity between the feature words and the window using the overlap coefficient is described by the following equation:

$$S_c = \frac{|T_G \cap T_w|}{T_G}, \tag{3}$$

Table 2. Data sets for our experiments.

Search-String			Ground Truth
Name	Num of Reviews	Lines of Reviews (Avg)	Lines (Sum)
Set 1	34	1.0	74
Set 2	30	3.5	626
Set 3	8	1.0	1069



Experimental results (Set 1). Experimental results (Set 2). Experimental results (Set 3).

Fig. 6. Experimental results of the first experiment. The Y-axis of all graphs represents the F-measure values. In almost all the range of the threshold, the accuracy using the feature words, labeled as "Characteristics Word", is higher than that without using the words.

where S_c is the similarity using feature words, and T_w is a set of words in a window. Finally, the similarity calculated using vectors of Word2Vec and the feature words are merged into a single value using the following formula: $S = \alpha * S_w + (1 - \alpha) * S_c$, where S represents the overall similarity ($0.0 \leq S \leq 1.0$), S_w is the similarity computed using Word2Vec ($0.0 \leq S_w \leq 1.0$), and α is a weight, a real number that falls within the range [0.0.1.0].

4.2. Evaluation

We evaluate accuracy and availability of the tool in detecting smells in specific categories in this section. The availability of the tool is assessed through two experiments, using three sets of reviews for search statements (Table 2). Set 1 comprises reviews classified under *access violation*, which includes reviews related to the assignment value of a read-only variable. Set 2 consists of 30 reviews randomly selected from the *substitutable process* category. Set 3 consists of reviews in the *inappropriate value of waiting time* category, containing a boundary value.

The goal of the first experiment is to measure the accuracy of the tool. Accuracy is measured using F-measure values. We measured the accuracy of smell detection using (1) Word2Vec, (2) feature words, and (3) both (Word2Vec and feature words). The second experiment examines the effectiveness of equivalence partitioning. We used Set 3 for the second experiment because members of Set 3 contain numerical values for equivalence partitioning. Common settings for two experiments are as follows: The tool was run using the three sets and their 76 bots. Bot used to generate search sentences was excluded from the bots used as the target bot (in Figure 3). The α value, responsible for merging the two types of similarity, was set to 0.75, determined by trials. Similarly, the θ_G value for deriving feature words was set to 0.75.

Figure 6 displays the results of the first experiment, evaluating the accuracy for three data sets. We observe that the proposed tool detected smells with high accuracy due to the relatively constrained nature of RPA code compared to conventional programming languages. In general coding, code descriptions fluctuate owing to programmers' coding styles. This fluctuation decreases the accuracy of the similarities. The standardized structure of RPA code, resulting from GUI-based bot development, on the other hand, minimizes coding style variation that can introduce noise, decreasing similarity accuracy. Thus, even with its simple mechanism, our preliminary tool delivers high accuracy.

Each combination of similarities yields interesting results. For Set 1 and Set 2, the F-measure values with both Word2Vec and feature words surpass those with one method alone for nearly all thresholds. For Set 3, the maximum value is also highest when both similarities are considered. This can be attributed to the high false positives problem encountered when using Word2Vec, mitigated by feature word similarity. Because the feature words must be exact matches of the words in the target code, the precision is high while the recall is low. Conversely, as Word2Vec generates

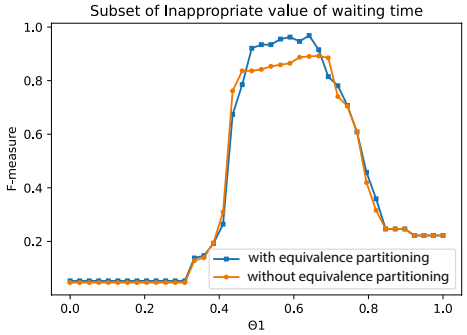


Fig. 7. Comparison of experimental results with and without the function of equivalence partitioning.

continuous values, the precision is lower while recall is higher. The complementary combination of these two aspects contributes to a high degree of accuracy.

We will now discuss the configuration of group sets and extraction of feature words. All members in Set 1 were grouped together, and feature words were correctly extracted. Extracted feature words included reserved words indicating read-only variables and variable assignments. In Set 2, some reviews were collated into a single group. These reviews pointed out the wait for window display in a loop. Reserved words related to loop processing and conditional statements, which verified whether a window exists or not, were extracted as feature words from this group. For Set 3, all elements were considered as belonging to a single group, leading to the extraction of primarily command names such as *Keystroke*, *delay*, and *in* as feature words. These feature words extracted from Sets 1 to 3 have strong connections to the smell: *assignment to a read-only variable*, *waiting for window display due to loop processing*, and *lengthen keystroke delay*, corresponding to the groups in Sets 1 to 3, respectively. These results demonstrate that our feature word extraction has been appropriate.

Figure 7 illustrates the results of the second experiment, which assesses the availability of equivalence partitioning. Notably, F-measure values with equivalence partitioning surpass those without it across almost the entire threshold range. This result shows its usefulness in detecting smells that necessitate numerical analysis, such as waiting time settings. Since smell detection associated with branching conditions and loop counts might also require numerical analysis, we can expect that our method can be applied to these smells. Our method assumes the existence of all appropriate boundary values in reviewed bots. Essentially, the method does not consider the magnitude of the value, implying that an excessively large or small value in the test cannot be detected unless a review points out the inappropriateness of the value. Therefore, the sufficiency of reviews is deemed a crucial factor when utilizing equivalence class partitioning.

Regarding parameter values, the α value was fixed at 0.75 in our experiments. However, the accuracy can be enhanced by setting an appropriate α for each category. For instance, the value of α should be raised from 0.75, when the entire code is related to a smell, such as the *substitutable process* category, and should be lowered from 0.75 when specific words are related to a smell, such as the *access violation* or *inappropriate argument* categories. As shown in Figure 6, the F-measure attains high values at high thresholds. This pattern could be attributed to the characteristics of codes including RPA codes and general programming languages. Codes tend to exhibit similar wordings and structures, even if they carry semantically different instructions. For example, two commands with identical argument structures differ by just one word in the command name, despite their execution details being different. Hence, when there are sentences that are semantically different but share similar words and structures, false positives increase, and precision decreases at lower thresholds. Therefore, it is prudent to set the threshold θ_{sm} to a higher value when the tool is applied to unknown data in actual operation.

5. Conclusions

This study established a hierarchical classification for identifying smells in RPA, using code reviews that highlight smells in actual bot development. We also developed a preliminary automatic detection tool for certain smells, utilizing Word2Vec, feature words, and equivalence class partitioning. Our categorization results yielded several in-

sights. Firstly, the prevalence of the *substitutable process* category underscored the importance of replacing a series of commands or a command for executing desktop tasks with a single RPA command. Users are not required to devise specific codes to handle desktop tasks because the RPA tool has predefined the most stable commands for these tasks. Secondly, categories unique to RPA, in comparison to those found in standard programs, emphasized the importance of interacting with external applications, effective error handling, and setting appropriate waiting times for commands. These measures are crucial to prevent defects caused by external applications running concurrently with bots. Additionally, the experimental results demonstrated that, despite its simple mechanism, the tool performed with high accuracy. The restriction on RPA code by the GUI programming leads to high accuracy.

The limitation of this study is that it analyzed only a particular RPA environment. Future research will aim to evaluate the applicability of the classification and the tool developed in this study to other RPA environments. Furthermore, enhancement of the smell detection tool will enable us to detect more smells across all categories.

References

[1] A.H.M. ter Hofstede A. Egger, S.J.J Leemans W. Kratsch, M. Röglinger, and M.T. Wynn, Bot Log Mining: Using Logs from Robotic Process Automation for Process Mining, In Conceptual Modeling, Springer International Publishing, pp. 51–61 (2020).

[2] R. Pereira F. Santos and J.B. Vasconcelos, Toward robotic process automation implementation: an end-to-end perspective, Business Process Management Journal 26, 2, pp. 405–420 (2020).

[3] P. Fersht, J.R. Slaby, Robotic automation emerges as a threat to traditional low-cost outsourcing. HfS Research (2012).

[4] Automation Anywhere Inc., Automation Anywhere, https://www.automationanywhere.com/products, (2024).

[5] D.S. Vugec, L. Ivančić, V.B. Vuksic, Robotic process automation: systematic literature review, In Business Process Management: Blockchain and Central and Eastern Europe Forum, Springer International Publishing, pp.280–295 (2019).
LSE Research Online Documents on Economics (2015).

[6] M.C. Lacity, L.P. Willcocks, A new approach to automating services, MIT Sloan Management Review (2017).

[7] G. Rothermel, M. Hammoudi, P. Tonella, Why do Record/Replay Tests of Web Applications Break?, In proc. of ICST 2016, pp. 180–190, IEEE (2016).

[8] N. Kolkin, M. Kusner, Y. Sun, K. Weinberger, From word embeddings to document distances, 37, pp. 957–966 (2015).

[9] R. Plattfaut, Robotic process automation - process optimization on steroids?, In Proc. of the 40th International Conference on Information Systems (ICIS'19), pp. 1-8 (2019).

[10] D.K. Jaiswal, S. Madakam, R. Holmukhe, The Future Digital Work Force: Robotic Process Automation (RPA), Journal of Information Systems and Technology Management 16, pp. 1–17 (2019).

[11] J. Schuler, F. Gehring, Implementing Robust and Low-Maintenance Robotic Process Automation (RPA) Solutions in Large Organisations, SSRN, pp. 1-29 (2018).

[12] M.K.Vijaymeena, K.Kavitha, A Survey on Similarity Measures in Text Mining, International Journal on Machine Learning and Applications (MLAIJ) Vol.3, No.1, pp. 19-28 (2016).

[13] G.S. Corrado, T. Mikolov, K. Chen, J. Dean, Efficient Estimation of Word Representations in Vector Space, ICLR Workshop Papers (2013).

[14] M. Bichler, W.M.P van der Aalst, A. Heinzl, Robotic Process Automation, Business and Information System Engineering 60, pp. 269–272 (2018).

[15] C. Cewe, D. Koch and R. Mertens, Minimal Effort Requirements Engineering for Robotic Process Automation with Test Driven Development and Screen Recording, Business Process Management Workshops, Springer, pp.642–648 (2017).

[16] C. Johannessen, T. Davenport, When Low-Code/No-Code Development Works — and When It Doesn't, Harvard Business Review (2021).

[17] P. Hallikainen, R. Bekkhus, S. L. Pan, How OpusCapita Used Internal RPA Capabilities to Offer Services to Clients, MIS Quarterly Executive, vol.17, no.1 (2018).

[18] D. Khramov, Robotic and Machine Learning: How to Help Support to Process Customer Tickets More Effectively, Bachelor's Thesis, Metropolia University of Applied Sciences (2018).

[19] R. Syed, S. Suriadi, M. Adams, W. Bandara, S.J. Leemans, C. Ouyang, A.H. Hofstede, I.V. Weerd, M.T. Wynn, and H.A. Reijers, Robotic Process Automation: Contemporary themes and challenges, Computers in Industry, Volume 115, Issue C (2020).

[20] Akimova, Elena N., Alexander Yu Bersenev, Artem A. Deikov, Konstantin S. Kobylkin, Anton V. Konygin, Ilya P. Mezentsev, and Vladimir E. Misilov. A Survey on Software Defect Prediction Using Deep Learning. Mathematics 9, no. 11, 1180 (2021).

[21] Y. Bengio, Learning Deep Architectures for AI. Found. Trends Mach. Learn., 2, 1–127 (2009).

[22] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning; MIT Press (2016).

[23] S. Hochreiter, J. Schmidhuber, Long Short-Term Memory., Neural Computation, 9, pp. 1735–1780 (1997).

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention Is All You Need (2017).

[25] R. Sindhgatta, A.H.M. ter Hofstede, A. Ghose, Resource-Based Adaptive Robotic Process Automation, In Proc. of CAiSE2020, LNCS, vol.12127, pp. 451–466 (2020).

[26] M. V. Mäntylä and C. Lassenius, What Types of Defects Are Really Discovered in Code Reviews?, IEEE Transactions on Software Engineering, vol.35, no.3, pp. 430-448 (2009).

[27] Matt J. Kusner, Yu Sun, Nicholas I. Kolkin, and Kilian Q. Weinberger, From word embeddings to document distances, In Proc. of the 32nd International Conference on International Conference on Machine Learning - Volume 37 (ICML'15), pp. 957–966 (2015).